TECHNICAL CURRICULUM & ARCHITECTURES

RAG, FAISS, Vector Databases & Knowledge Systems

A Comprehensive Technical Guide to Vector Search, Indexing & Enterprise Retrieval

1. Foundations of Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) grounds Large Language Models (LLMs) on external, dynamic, and domain-specific knowledge bases. By combining dense vector retrieval with autoregressive generation, RAG mitigates hallucinations, enforces factual alignment, and overcomes static model knowledge cutoff constraints.

This technical guide covers document ingestion, vector similarity mechanics, high-dimensional indexing algorithms (FAISS, HNSW, IVF-PQ), hybrid search paradigms, advanced retrieval strategies, and complete vector knowledge system implementations.

> **The RAG Optimization Equation**
> A RAG pipeline optimizes the conditional generation probability $P(Y \mid X) = \sum_{d \in D} P(Y \mid X, d)\, P(d \mid X)$, where candidate document chunks d are retrieved from corpus D based on query relevance $P(d \mid X)$ before conditioning output sequence generation Y.

2. Document Ingestion & Chunking Strategies

Raw documents must be segmented into semantically coherent textual units (chunks) before vector embedding generation. Chunking directly influences context window granularity and retrieval precision.

2.1 Chunking Taxonomy

Strategy	Mechanism	Advantages	Trade-offs
Fixed-Size Chunking	Splits by static character/token counts with overlay sliding windows	Simple, deterministic, fast compute	Breaks sentences mid-concept; lacks semantic awareness
Recursive Character	Hierarchical splitting using delimiters (`\n\n` , `\n` , `.` , `" "`)	Preserves structural boundaries and paragraphs	May vary in token size across distinct document types
Semantic Chunking	Computes sentence embedding similarity distances to identify topic shifts	High semantic boundary fidelity	Computationally expensive; relies on embedding model quality
Parent-Document / Hierarchical	Embeds small child chunks for retrieval, returns larger parent block to LLM	Precise vector matches + complete context payload	Requires dual-level storage schema indexing

2.2 Recursive Chunking Implementation

Below is a Python implementation of recursive sliding-window text chunking with token-count bounding.

```
def recursive_text_chunker(text: str, max_chunk_size: int = 200, overlap: int = 40) ->
list[str]:
    """Splits text recursively using structural separators while keeping token overlap."""
    separators = ["\n\n", "\n", ". ", " ", ""]

    def _split(doc: str, sep_idx: int) -> list[str]:
        if len(doc) <= max_chunk_size or sep_idx >= len(separators):
            return [doc] if doc.strip() else []

        sep = separators[sep_idx]
        splits = doc.split(sep) if sep != "" else list(doc)

        chunks = []
        current = ""
        for s in splits:
            candidate = current + (sep if current else "") + s
            if len(candidate) <= max_chunk_size:
                current = candidate
            else:
                if current:
                    chunks.append(current)
                current = s
        if current:
            chunks.append(current)

        # Recursive pass for chunks that still exceed maximum size
        final_chunks = []
        for c in chunks:
            if len(c) > max_chunk_size:
                final_chunks.extend(_split(c, sep_idx + 1))
            else:
                final_chunks.append(c)
        return final_chunks

    raw_chunks = _split(text, 0)
    return raw_chunks

# Sample Execution
document = "RAG architectures connect LLMs to vector DBs.\n\nIndexing enables fast
similarity search over millions of chunks."
processed_chunks = recursive_text_chunker(document, max_chunk_size=60, overlap=15)
print(f"Generated {len(processed_chunks)} chunks:", processed_chunks)
```

3. Vector Embeddings & Distance Metrics

Dense embedding models project unstructured textual chunks into continuous vector spaces $\mathbb{R}^d$ (typically $d \in [384, 1536]$), preserving semantic relationships.

3.1 High-Dimensional Distance Formulations

Metric	Mathematical Formula	Properties & Optimal Context
Cosine Similarity	$cos(u, v) = (u \cdot v) / (\|\|u\|\| \|\|v\|\|)$	Scale-invariant angle metric; standard for text embeddings. Range: [-1, 1].
Dot Product (Inner Product)	$IP(u, v) = \sum u_i v_i = u \cdot v$	Identical to Cosine Similarity when vectors are L2-normalized ($\|\|u\|\| = \|\|v\|\| = 1$). Compute-efficient.
Euclidean Distance (L2)	$d_{L2}(u, v) = \sqrt{(\sum (u_i - v_i)^2)}$	Measures spatial distance. Sensitive to vector magnitude differences. Range: [0, ∞).

3.2 Vectorized Distance Calculation Engine

```
import numpy as np

def compute_similarity_scores(query_vec: np.ndarray, doc_matrix: np.ndarray, metric: str =
"cosine") -> np.ndarray:
    """Vectorized distance computation between 1D query vector and 2D matrix of
embeddings."""
    if metric == "dot":
        return np.dot(doc_matrix, query_vec)

    elif metric == "cosine":
        norm_q = np.linalg.norm(query_vec)
        norm_docs = np.linalg.norm(doc_matrix, axis=1)
        # Prevent division by zero
        norm_docs[norm_docs == 0] = 1e-10
        norm_q = 1e-10 if norm_q == 0 else norm_q
        return np.dot(doc_matrix, query_vec) / (norm_docs * norm_q)

    elif metric == "l2":
        # Returns negative L2 distance so higher value = closer vector
        diff = doc_matrix - query_vec
        return -np.sqrt(np.sum(diff**2, axis=1))

    else:
        raise ValueError(f"Unsupported metric: {metric}")

# Example Demonstration
query = np.array([0.1, 0.8, -0.4])
docs = np.array([[0.2, 0.7, -0.3], [-0.5, 0.1, 0.8], [0.1, 0.8, -0.4]])
scores = compute_similarity_scores(query, docs, metric="cosine")
print("Cosine Similarities:", np.round(scores, 4))
```

> **Embedding Dimensionality vs. Index Performance**
> Doubling vector dimensionality d doubles storage footprint and dot-product compute overhead. Matryoshka Representation Learning (MRL) enables slicing embedding dimensions (e.g., 1536 → 256) while retaining >90% retrieval accuracy.

4. High-Performance Vector Indexing (FAISS & Beyond)

Exhaustive exact search ($O(N \cdot d)$) becomes unscalable for millions of vectors. Approximate Nearest Neighbor (ANN) indexes trade marginal recall accuracy for orders-of-magnitude faster queries ($O(\log N)$ or $O(1)$).

4.1 Modern Indexing Paradigms

Index Structure	Underlying Data Structure	Search Speed	Memory Consumption
Flat (IndexFlatIP / L2)	Raw dense matrix array	$O(N)$ (Brute-force)	Baseline ($N \times d \times 4$ bytes)
Inverted File (IVF)	Voronoi cell partitioning via k-means centroids	Sub-linear ($O(nprobe \times N / nlist)$)	Low additional overhead
Product Quantization (PQ)	Sub-vector codebook quantization to byte indices	Fast distance lookup tables	Extremely Low (up to 95% reduction)
HNSW (Hierarchical Navigable Small World)	Multi-layer proximity graph network	Ultra-fast ($O(\log N)$)	High (Stores graph edges + vectors)

4.2 Inverted File Index (IVF) Mechanics Simulation

Below is a pure Python implementation of an IVF Index demonstrating cluster centroid routing.

```
class SimpleIVFIndex:
    """Simplified Inverted File (IVF) index structure for ANN vector search."""
    def __init__(self, nlist: int = 4):
        self.nlist = nlist
        self.centroids = None
        self.inverted_lists = {}

    def fit(self, vectors: np.ndarray):
        """Trains cluster centroids using naive k-means initialization."""
        indices = np.random.choice(len(vectors), self.nlist, replace=False)
        self.centroids = vectors[indices]
        for i in range(self.nlist):
            self.inverted_lists[i] = []

        # Assign training vectors to nearest centroid
        for idx, vec in enumerate(vectors):
            dists = np.linalg.norm(self.centroids - vec, axis=1)
            best_cluster = int(np.argmin(dists))
            self.inverted_lists[best_cluster].append((idx, vec))

    def search(self, query: np.ndarray, top_k: int = 2, nprobe: int = 2) ->
list[tuple[int, float]]:
        """Searches only the top-nprobe nearest centroid clusters."""
        centroid_dists = np.linalg.norm(self.centroids - query, axis=1)
        closest_clusters = np.argsort(centroid_dists)[:nprobe]

        candidates = []
        for cluster_id in closest_clusters:
            for vec_id, vec in self.inverted_lists[cluster_id]:
                sim = np.dot(query, vec) / (np.linalg.norm(query) * np.linalg.norm(vec))
                candidates.append((vec_id, float(sim)))

        candidates.sort(key=lambda x: x[1], reverse=True)
        return candidates[:top_k]

# Test IVF Index Search
np.random.seed(42)
dataset = np.random.randn(100, 8)
ivf = SimpleIVFIndex(nlist=5)
ivf.fit(dataset)
results = ivf.search(dataset[0], top_k=3, nprobe=2)
print("IVF Top Matches (ID, Cosine Score):", results)
```

5. Hybrid Search Architecture & Sparse-Dense Fusion

Dense vector search excels at conceptual semantic matching, but can fail on exact keyphrase identifiers, codes, or domain terms. Hybrid search fuses dense embeddings with sparse lexical representations (BM25).

5.1 Lexical vs. Dense Retrieval Models

Dimension	Sparse Lexical (BM25)	Dense Vector (Embeddings)
Matching Target	Exact terms, keyword frequencies, inverted index lookup	Latent semantic meaning, context, synonyms
Failure Modes	Vocabulary mismatch / synonymy failure	Hallucinated relevance on exact part numbers or rare named entities
Scoring Metric	TF-IDF with document length normalization	Cosine Similarity / Inner Product in embedding space

5.2 Reciprocal Rank Fusion (RRF) Algorithm

RRF merges rank lists from distinct retrieval engines without requiring score normalization calibration across disparate scales:

$RRF_Score(d \in D) = \sum_{m \in Models} 1 / (k + r_m(d))$ where default penalty constant $k = 60$.

```
def reciprocal_rank_fusion(rank_lists: list[list[int]], k: int = 60) -> list[tuple[int, float]]:
    """Combines multiple ranked document ID lists using Reciprocal Rank Fusion."""
    rrf_scores = {}
    for r_list in rank_lists:
        for rank, doc_id in enumerate(r_list, start=1):
            if doc_id not in rrf_scores:
                rrf_scores[doc_id] = 0.0
            rrf_scores[doc_id] += 1.0 / (k + rank)
                
    sorted_docs = sorted(rrf_scores.items(), key=lambda x: x[1], reverse=True)
    return sorted_docs

# Example RRF Fusion Execution
dense_ranks = [101, 104, 102, 108]
sparse_ranks = [102, 101, 109, 104]
fused_results = reciprocal_rank_fusion([dense_ranks, sparse_ranks], k=60)
print("RRF Fused Order (DocID, Score):", fused_results)
```

6. Reranking & Multi-Stage Retrieval Pipelines

Retrieval pipelines employ a multi-stage architecture: a high-throughput vector retriever fetches candidate documents (e.g., Top-100), followed by a computationally heavier Cross-Encoder reranker to score candidate relevance accurately.

6.1 Bi-Encoders vs. Cross-Encoders

Architecture Type	Computation Strategy	Accuracy	Latency
Bi-Encoder (Retriever)	Embeds Query and Document independently; computes dot-product vector distance	Moderate (Independent representation)	Sub-millisecond (Pre-indexed vectors)
Cross-Encoder (Reranker)	Feeds Query + Document jointly through full transformer self-attention layers	State-of-the-art (Full joint interaction)	Higher (Tens of milliseconds per pair)

6.2 Cross-Encoder Reranking Simulation

```
class DummyCrossEncoderReranker:
    """Simulates Cross-Encoder joint interaction scoring between query and retrieved chunks."""
    def rerank(self, query: str, candidate_chunks: list[str], top_n: int = 2) -> list[tuple[str, float]]:
        scores = []
        query_words = set(query.lower().split())
        for chunk in candidate_chunks:
            chunk_words = set(chunk.lower().split())
            # Simulated joint self-attention score (Jaccard + length penalty)
            intersection = len(query_words.intersection(chunk_words))
            union = len(query_words.union(chunk_words))
            score = intersection / union if union > 0 else 0.0
            scores.append((chunk, score))

        scores.sort(key=lambda x: x[1], reverse=True)
        return scores[:top_n]

# Test Reranking Stage
candidates = [
    "Vector databases store high dimensional embeddings.",
    "Python is a versatile programming language for data science.",
    "FAISS indexes support fast vector search over dense embeddings."
]
reranker = DummyCrossEncoderReranker()
reranked = reranker.rerank("vector databases FAISS search", candidates, top_n=2)
print("Reranked Top Chunks:", reranked)
```

7. Query Transformation & Context Expansion Techniques

Raw user queries are often ambiguous, underspecified, or keyword-deficient. Advanced RAG workflows apply query transformation pipelines before executing vector search.

7.1 Query Enhancement Taxonomy

- **Query Rewriting:** Standardizes input queries by correcting misspellings, removing conversational noise, and injecting domain terms.
- **Query Decomposition:** Breaks down complex multi-hop queries into sub-questions executed sequentially or in parallel against the index.
- **Sub-Query Generation (RAG-Fusion):** Expands a single user prompt into 3–5 diverse variations to query the vector database from multiple semantic angles.
- **HyDE (Hypothetical Document Embeddings):** Uses an LLM to generate a zero-shot hypothetical answer document, embedding the answer instead of the query for semantic retrieval.

7.2 HyDE Query Transformation Pipeline

```
def generate_hypothetical_document(query: str) -> str:
    """Simulates LLM generating a hypothetical answer document for HyDE retrieval."""
    # In production, this calls an LLM API: prompt = f"Write a passage answering: {query}"
    hypothetical_answer = (
        f"Hierarchical Navigable Small World (HNSW) graphs solve high-dimensional vector search "
        f"by constructing multi-layer graph structures where top layers route coarse jumps "
        f"and bottom layers execute localized graph traversal."
    )
    return hypothetical_answer

def hyde_retrieval_flow(user_query: str, vector_store_fn) -> list[str]:
    """Executes HyDE retrieval by embedding the generated hypothetical passage."""
    hypo_doc = generate_hypothetical_document(user_query)
    # Search vector index using hypothetical document string representation
    retrieved_chunks = vector_store_fn(hypo_doc)
    return retrieved_chunks

# HyDE Flow Dry-Run
results = hyde_retrieval_flow("How does HNSW graph search work?", lambda doc: [f"Matched chunk via HyDE text: {doc[:60]}..."])
print("HyDE Execution Result:", results)
```

8. Vector Database Production Architectures

Production enterprise vector databases manage persistence, multi-tenancy, filtering, replication, and CRUD vector lifecycle operations at scale.

8.1 Vector DB Engine Comparison

Vector DB Engine	Architecture Type	Supported Indexes	Filtering Paradigm
Pinecone	Fully-managed cloud native serverless	Proprietary graph / HNSW variants	Single-stage metadata filtering
Milvus	Distributed cloud-native microservices (Go/C++)	HNSW, IVF-PQ, ScaNN, GPU indexes	Iterative scalar-filtered vector search
Qdrant	Self-hosted / Cloud (Rust)	Custom HNSW graph with payload index	Payload-based pre-filtering on graph links
pgvector	PostgreSQL extension	IVFFlat, HNSW	SQL relational JOINs + WHERE clause integration

8.2 Metadata Pre-Filtering vs. Post-Filtering

> **The Single-Stage Filter Advantage**
> Post-filtering retrieves Top-K vectors first, then discards non-matching metadata rows, risking return of zero results. Pre-filtering restricts graph or IVF traversal exclusively to vectors passing metadata predicates.

9. End-to-End Modular Vector Knowledge System

Below is a complete, runnable Python implementation integrating text chunking, embedding generation simulation, IVF vector indexing, BM25 sparse search, RRF fusion, and prompt context building.

```python
class VectorKnowledgeSystem:
    """Complete end-to-end RAG knowledge pipeline implementation in Python."""
    def __init__(self, embed_dim: int = 16):
        self.embed_dim = embed_dim
        self.documents = []
        self.doc_vectors = None

    def _embed(self, text: str) -> np.ndarray:
        """Deterministic vector embedding generation based on character hash seeds."""
        rng = np.random.RandomState(abs(hash(text)) % (2**32))
        vec = rng.randn(self.embed_dim)
        return vec / np.linalg.norm(vec)

    def ingest_documents(self, text_list: list[str]):
        """Ingests text chunks into internal vector storage matrix."""
        self.documents = text_list
        matrix = [self._embed(doc) for doc in text_list]
        self.doc_vectors = np.array(matrix)

    def vector_search(self, query: str, top_k: int = 3) -> list[tuple[int, float]]:
        """Computes cosine vector retrieval scores."""
        q_vec = self._embed(query)
        scores = np.dot(self.doc_vectors, q_vec)
        ranked_indices = np.argsort(scores)[::-1][:top_k]
        return [(idx, float(scores[idx])) for idx in ranked_indices]

    def build_rag_prompt(self, query: str, top_k: int = 2) -> str:
        """Constructs augmented LLM prompt context from top retrieved passages."""
        search_results = self.vector_search(query, top_k=top_k)
        retrieved_contexts = [self.documents[idx] for idx, _ in search_results]

        context_str = "\n---\n".join(retrieved_contexts)
        prompt = (
            f"[SYSTEM PROMPT: Answer query strictly using retrieved context below.]\n\n"
            f"### CONTEXT:\n{context_str}\n\n"
            f"### QUERY: {query}\n\n"
            f"### ANSWER:"
        )
        return prompt

# Complete Knowledge Pipeline Dry Run
if __name__ == "__main__":
    corpus = [
        "Retrieval-Augmented Generation (RAG) grounds LLM responses using external
context.",
        "FAISS provides high performance approximate nearest neighbor vector indexing.",
        "Product Quantization reduces vector memory overhead by compressing float
vectors.",
        "BM25 is a keyword-based sparse retrieval model utilizing term frequency scoring."
    ]

    rag_system = VectorKnowledgeSystem(embed_dim=8)
    rag_system.ingest_documents(corpus)

    user_query = "What is FAISS used for in vector search?"
    generated_prompt = rag_system.build_rag_prompt(user_query, top_k=2)
    print("--- Generated RAG Prompt Execution Output ---\n")
```

10. RAG & Vector System Architecture Summary

Core Knowledge System Engineering Pillars

- **Ingestion & Chunking:** Semantic & recursive chunking preserve document boundaries and context payload integrity.
- **Indexing Mechanics:** HNSW (graph) and IVF-PQ (quantized centroids) balance search speed against memory consumption.
- **Retrieval Synergy:** Hybrid search (BM25 + Dense Vectors) fused via Reciprocal Rank Fusion (RRF) eliminates single-model retrieval gaps.
- **Multi-Stage Precision:** Bi-Encoder fast retrieval followed by Cross-Encoder reranking maximizes precision within context limits.